Autonomous Movement

1. Purpose of the code

This code is a **first-stage autonomous target approach tester** for your ROV.

It does the following:

1. reads an image, image folder, video, or webcam
2. uses YOLO to detect visible objects
3. selects one target automatically
4. calculates the target position relative to the center of the frame
5. decides how the ROV should move:
    - rotate left/right
    - move up/down
    - move forward
6. prints the motion command that can later be sent to the software/control team

Important:

- this version does **not control the real ROV directly**
- it is meant for **testing and integration**
- it chooses the **largest detected object**
- it does **not depend on object class name**

2. What you need before running it

You need these things installed on your PC.

A. Python

Install Python 3.10 or newer.

Check with:

```
python --version
```

or:

```
py --version
```

B. VS Code

Install Visual Studio Code.

Also install the **Python extension** in VS Code.

C. Required Python libraries

Open terminal in VS Code and run:

```
pip install ultralytics opencv-python
```

If `pip` does not work, try:

```
python -m pip install ultralytics opencv-python
```

3. Files you need

Your folder can look like this:

```
ROV_Project/
│
├── rov_autonomous_test.py
├── underwater.mp4
├── test.jpg
├── test_images/
│    ├── img1.jpg
│    ├── img2.jpg
│    └── img3.jpg
```

You do **not** need to manually download `yolov8s.pt` first.
When you run:

```
model = YOLO(“yolov8s.pt")
```

Ultralytics usually downloads it automatically the first time.

If internet is blocked, then you can manually download the model once and place it in the same folder.

4. What the script uses internally

The script uses:

- **YOLOv8s pretrained model**
- **largest bounding box** as the target
- **state machine** for motion decision
- **smoothed position values** to reduce noise

5. Main idea of operation

The script works like this:

```
Input frame
   -> YOLO detects objects
   -> choose largest object
   -> compute x_error, y_error, area
   -> decide state
   -> output movement command
```

6. Meaning of the states

The code uses 4 states.

SEARCH

The system is searching for a stable target.

Behavior:

- no forward motion
- slow yaw rotation
- waits until object appears consistently

Typical command:

```
{
    "surge": 0.0,
    "heave": 0.0,
    "yaw": 0.15
}
```

Meaning:

- rotate slowly to scan environment

ALIGN

The system has found a target and tries to center it.

Behavior:

- correct left/right using `yaw`
- correct up/down using `heave`
- no forward movement yet

Typical command:

```
{
    "surge": 0.0,
    "heave": -0.12,
    "yaw": 0.08
}
```

Meaning:

- rotate right a little
- move upward a little

APPROACH

The target is centered enough, so the system moves toward it.

Behavior:

- move forward with `surge`
- continue correcting yaw/heave while approaching

Typical command:

```
{
    "surge": 0.19,
    "heave": -0.03,
    "yaw": 0.05
}
```

Meaning:

- move forward
- small alignment correction

HANDOVER

The target is considered close enough.

Behavior:

- stop autonomous motion
- hand control to operator

Typical command:

```
{
    "surge": 0.0,
    "heave": 0.0,
    "yaw": 0.0
}
```

Meaning:

- stop
- ready for manual gripping

7. Meaning of the important variables

`x_error`

Horizontal target error from image center.

Formula:

```
x_error = target_x - 0.5
```

Interpretation:

- negative -> target is on the left
- positive -> target is on the right
- near zero -> target is centered horizontally

Example:

```
x_error = -0.20
```

Meaning:

- target is clearly left of center

`y_error`

Vertical target error from image center.

Formula:

```
y_error = target_y - 0.5
```

Interpretation:

- negative -> target is above center
- positive -> target is below center
- near zero -> target is centered vertically

Example:

```
y_error = 0.14
```

Meaning:

- target is below center

area

Normalized area of the bounding box.

Formula:

```
area = bbox_width * bbox_height
```

Interpretation:

- small area -> target is far
- large area -> target is close

Example:

```
area = 0.02
```

Meaning:

- target is still far

Example:

```
area = 0.12
```

Meaning:

- target is much closer

surge

Forward/backward motion command.

In your current logic:

- positive surge -> move forward
- zero -> no forward motion

heave

Vertical motion command.

In your current code:

- positive -> move down
- negative -> move up

This must be confirmed with software/control team because sign convention may differ.

`yaw`

Rotation command.

In your current code:

- positive -> rotate right
- negative -> rotate left

Also must be confirmed with software/control team.

8. How to run the code

A. Test one image

Set:

```
MODE = "image"
IMAGE_PATH = "test.jpg"
```

Run:

```
python rov_autonomous_test.py
```

What happens:

- script opens one image
- detects objects
- chooses target
- shows result window
- prints one output in terminal

B. Test a folder of images

Set:

```
MODE = "images"
IMAGES_FOLDER = "test_images"
```

Run:

```
python rov_autonomous_test.py
```

What happens:

- script reads images one by one
- shows each result
- press any key for next image
- press `q` or `Esc` to stop

C. Test a video

Set:

```
MODE = "video"
VIDEO_PATH = "underwater.mp4"
```

Run:

```
python rov_autonomous_test.py
```

What happens:

- script processes video frame by frame
- shows live autonomous decisions
- prints output continuously

D. Test a webcam

Set:

```
MODE = "webcam"
WEBCAM_INDEX = 0
```

Run:

```
python rov_autonomous_test.py
```

What happens:

- reads webcam frames in real time
- detects object
- prints commands live

9. How to stop the program

Press:

```
q
```

or:

```
Esc
```

10. What appears on the screen

The output window shows:

- green rectangle around selected target
- blue point = image center
- red point = target center
- line between image center and target center
- current state
- current action
- motion command values

This helps you visually verify whether the command makes sense.

11. Example terminal outputs and their meaning

Example 1

```
{
    'target_detected': True,
    'state': 'SEARCH',
    'action': 'search_scan',
    'command': {'surge': 0.0, 'heave': 0.0, 'yaw': 0.15},
    'target': {'confidence': 0.757, 'x_error': -0.028,
'y_error': -0.18, 'area': 0.1512}
}
```

Meaning:

- target is detected
- system is still confirming stable detection
- no forward movement yet
- slow right scan is active
- object is slightly left and above center
- object already appears fairly large

Example 2

```
{
    'target_detected': True,
    'state': 'ALIGN',
    'action': 'aligning',
    'command': {'surge': 0.0, 'heave': -0.22, 'yaw': 0.17},
    'target': {'confidence': 0.843, 'x_error': 0.145,
'y_error': -0.181, 'area': 0.0415}
}
```

Meaning:

- target is stable
- system is aligning
- target is on the right -> positive yaw
- target is above -> negative heave
- no forward motion yet because it is still centering

Example 3

```
{
    'target_detected': True,
    'state': 'APPROACH',
    'action': 'approaching',
    'command': {'surge': 0.231, 'heave': -0.04, 'yaw': 0.03},
    'target': {'confidence': 0.901, 'x_error': 0.026,
'y_error': -0.031, 'area': 0.0563}
}
```

Meaning:

- target is centered enough
- system starts moving forward
- small corrections remain
- object is still not close enough, so continue approach

Example 4

```
{
    'target_detected': True,
    'state': 'HANDOVER',
    'action': 'close_enough_handover',
    'command': {'surge': 0.0, 'heave': 0.0, 'yaw': 0.0},
    'target': {'confidence': 0.933, 'x_error': 0.011,
'y_error': -0.015, 'area': 0.1128}
}
```

Meaning:

- target is close enough
- stop autonomous motion
- ready to switch to manual gripping

Example 5

```
{
    'target_detected': False,
    'state': 'SEARCH',
    'action': 'search_scan_no_target',
    'command': {'surge': 0.0, 'heave': 0.0, 'yaw': 0.15}
}
```

Meaning:

- no object detected
- keep scanning

12. What should be sent to the software team

The software team mainly needs the **motion command** and optionally the target info.

Best minimal format to send:

```
{
    "state": "APPROACH",
    "action": "approaching",
    "command": {
        "surge": 0.231,
        "heave": -0.04,
        "yaw": 0.03
    }
}
```

This is enough if they only care about motion.

13. Better format to send to the software team

Use this full format:

```
{
    "target_detected": True,
    "state": "APPROACH",
    "action": "approaching",
    "command": {
        "surge": 0.231,
        "heave": -0.04,
        "yaw": 0.03
    },
```

```
    "target": {
        "confidence": 0.901,
        "x_error": 0.026,
        "y_error": -0.031,
        "area": 0.0563
    }
}
```

Why this is better:

- software team gets motion command
- they also get vision status
- easier to debug integration

14. What the software team should understand from each field

`target_detected`

- `True` -> target currently exists
- `False` -> no target right now

`state`

Current autonomy phase:

- `SEARCH`
- `ALIGN`
- `APPROACH`
- `HANDOVER`

`action`

Readable explanation of what the autonomy is doing:

- `search_scan`
- `aligning`
- `approaching`
- `handover_to_operator`

`command.surge`

Forward motion request

`command.heave`

Vertical motion request

`command.yaw`

Rotation motion request

`target.x_error`

Horizontal alignment error

`target.y_error`

Vertical alignment error

`target.area`

Approximate closeness indicator

15. Very important sign-convention agreement with software team

Before integration, you must confirm these meanings.

In your current code, assumed meaning is:

```
surge > 0  -> forward
heave > 0  -> down
heave < 0  -> up
yaw > 0    -> right
yaw < 0    -> left
```

The software team must confirm whether their controller uses the same signs.

If not, you must invert the values before sending.

Example:
if their positive yaw means left, then send:

```
yaw_to_send = -yaw
```

16. Message to send to software team

The autonomous vision module outputs one command packet per frame.

Fields:

- target_detected: bool
- state: SEARCH / ALIGN / APPROACH / HANDOVER
- action: readable action string
- command:
  - surge: forward command
  - heave: vertical command
  - yaw: rotation command
- target:
  - confidence
  - x_error
  - y_error
  - area

Current sign convention assumed by AI module:

- surge > 0 = forward
- heave > 0 = down
- yaw > 0 = right

Example packet:

{
"target_detected": true,
"state": "APPROACH",
"action": "approaching",
"command": {
"surge": 0.231,
"heave": -0.04,
"yaw": 0.03
},
"target": {
"confidence": 0.901,
"x_error": 0.026,
"y_error": -0.031,
"area": 0.0563
}
}

17. Parameters you will probably tune during testing

CONF_THRESHOLD

Default:

```
CONF_THRESHOLD = 0.4
```

Increase it if there are too many false detections.

Decrease it if detection is weak.

REQUIRED_STABLE_FRAMES

Default:

```
REQUIRED_STABLE_FRAMES = 3
```

Higher value:

- safer
- slower to react

Lower value:

- faster
- more sensitive to noise

X_THRESHOLD and Y_THRESHOLD

These control how centered the target must be before moving forward.

Smaller threshold:

- more accurate alignment
- may take longer

Larger threshold:

- faster
- less precise

AREA_CLOSE_THRESHOLD

This controls when the system stops and hands over.

Smaller value:

- handover earlier
- farther from object

Larger value:

- handover later
- closer to object

`K_YAW`, `K_HEAVE`, `K_SURGE`

These are control gains.

Higher gain:

- faster movement
- more aggressive

Lower gain:

- smoother
- slower

18. Common problems and fixes

Problem: no object detected

Possible reasons:

- input path wrong
- model not loaded
- underwater object not recognized by pretrained YOLO
- confidence too high

Fixes:

- check path
- lower `CONF_THRESHOLD`
- test on easier images first

Problem: wrong object is selected

Cause:

- script chooses largest detected object

Fix:

- later you can add class filtering
- or choose center-most object instead of largest

Problem: movement commands look unstable

Cause:

- noisy detections

Fix:

- increase smoothing

- increase stable frames
- lower gains

Problem: ROV would rotate the wrong direction

Cause:

- sign convention mismatch

Fix:

- invert `yaw` or `heave` before sending to team

Problem: it stops too early or too late

Cause:

- `AREA_CLOSE_THRESHOLD` not tuned

Fix:

- change this value based on object size and camera distance

19. Current limitations of this code

This version is good for testing, but it still has limitations:

- pretrained YOLO may misclassify underwater objects
- no custom underwater training yet
- no real depth sensor
- distance is estimated only from bounding box area
- no direct thruster control yet
- no obstacle avoidance

That is normal for a first integration phase.

20. Best next step after this

After this script works well on underwater images/video, do this next:

1. define exact packet format with software team
2. send commands directly by UDP/serial/ROS
3. test sign conventions
4. run pool tests with low gains
5. later train YOLO on your real underwater object data